

12강

spring Boot

MVC 회원 로그인



비밀번호 암호화

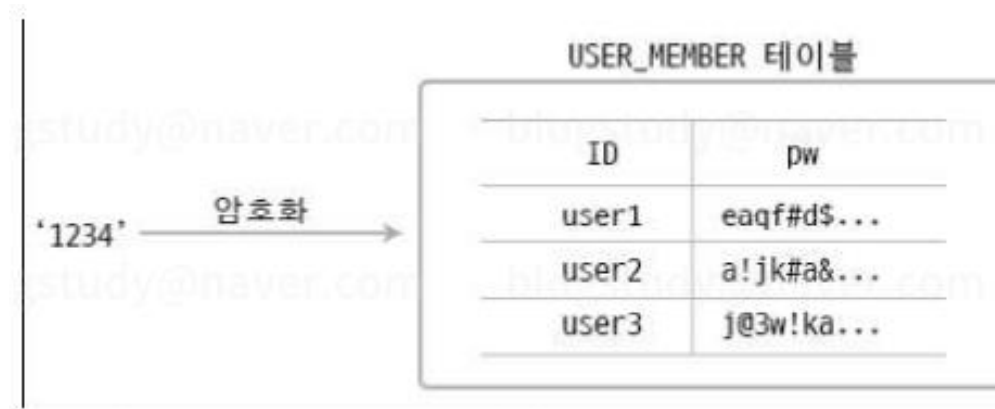




spring

➤ 회원가입 시 비밀번호를 암호화 하라

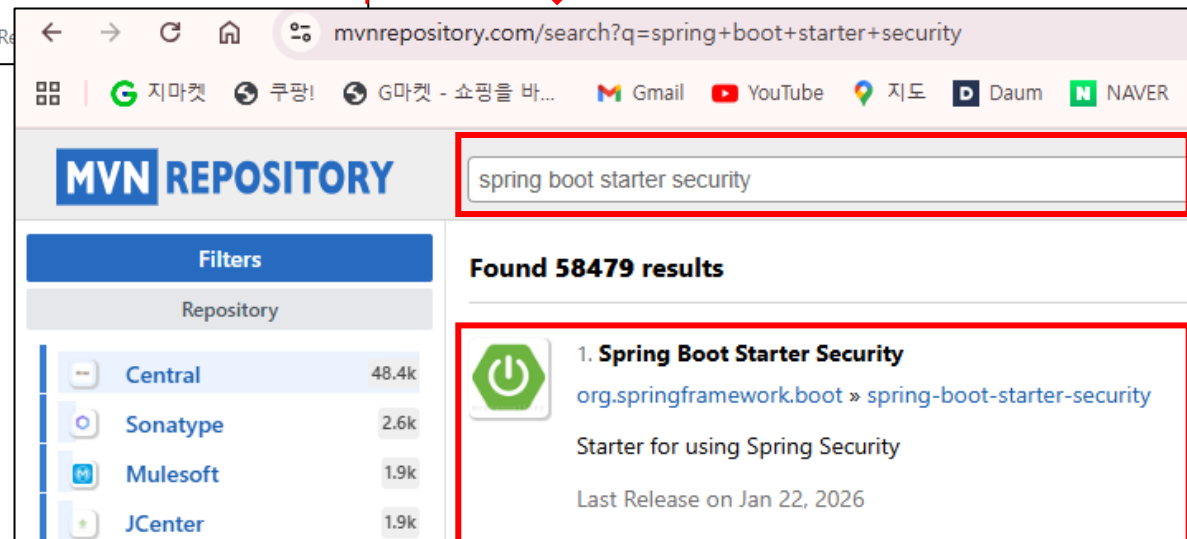
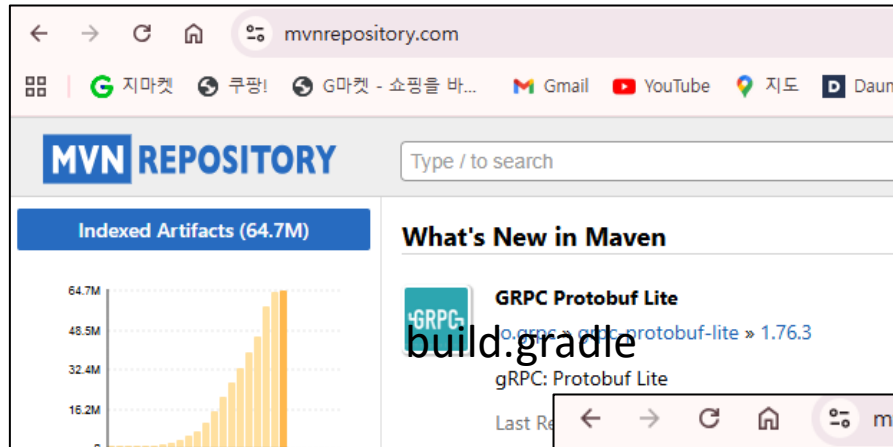
MemberDao의 signupConfirm()에는보안상문제가있습니다. 만약 사용자가 비밀번호로 '1234'를입력하면 'PW' 컬럼에 '1234'가들어 갑니다. 그런데 비밀번호는 다른 사람이 알아서는안되는 중요한 정보로, DB에접근 가능한 조직의 내부 인력 누구도 다른사람의 비밀번호를 확인할수 있으면 안됩니다. 따라서 비밀번호와 같은 중요한 정보는 데이터베이스에 암호화해야 합니다.





spring

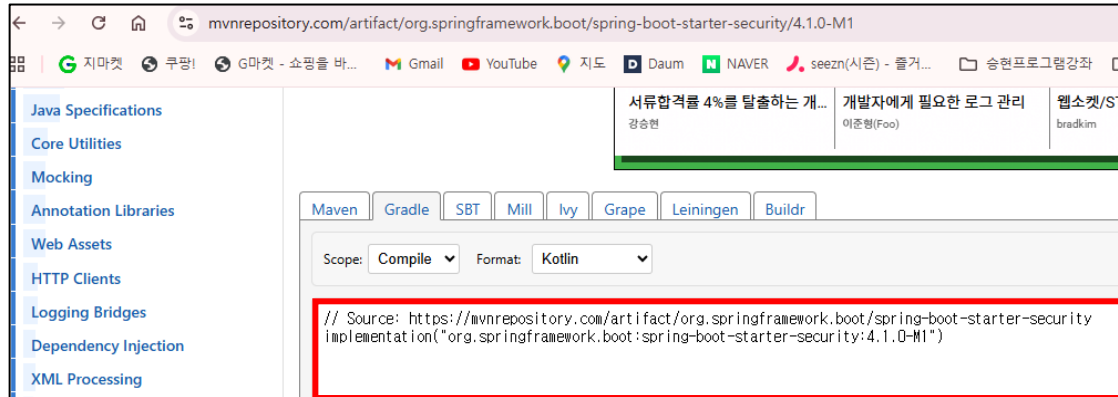
- 회원가입 시 비밀번호를 암호화 하라
 - security 의존성을 프로젝트에 추가한다.





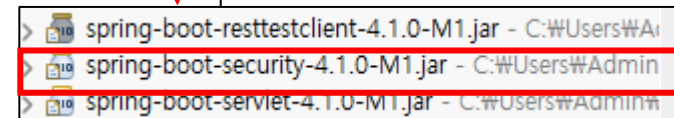
spring

- 회원가입 시 비밀번호를 암호화 하라
 - **build.gradle** 에 Security를 추가한다.



반드시 gradle -> Refresh gradle project 한다.

```
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'  
    implementation 'org.springframework.boot:spring-boot-starter-web' // * 변경  
    //implementation 'org.springframework.boot:spring-boot-starter-webmvc'  
  
    // Source: https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-jdbc  
    implementation("org.springframework.boot:spring-boot-starter-jdbc")  
  
    // Source: https://mvnrepository.com/artifact/com.mysql/mysql-connector-j  
    implementation("com.mysql:mysql-connector-j")  
  
    // Source: https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-security  
    implementation("org.springframework.boot:spring-boot-starter-security")  
  
    developmentOnly 'org.springframework.boot:spring-boot-devtools'  
    testImplementation 'org.springframework.boot:spring-boot-starter-thymeleaf-test'  
    testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'  
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'  
}
```





spring

- 회원가입 시 **비밀번호**를 암호화 하라
 - Spring Security 설정 위해서 config 패키지를 만들고, SecurityConfig 클래스를 만든다.



- MemberService에 멤버필드로 PasswordEncoder를 선언하고 의존 객체를 자동 주입 한다.

```
package com.green.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

//Spring Security 설정을 위해서 config 패키지를 만들고, SecurityConfig클래스를 만든다.

@Configuration
@EnableWebSecurity //우리가 지정한 암호화를 웹어플리케이션에 적용하겠다는 어노테이션
public class SecurityConfig {

    @Bean // Ioc컨테이너에 Bean으로 등록한다.
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(); //문자열을 암호화 해주는 클래스이다.
    }
}
```



spring

- 회원가입 시 비밀번호를 암호화 하라
- **package** com.green.config 안에 아래 코드를 복사 붙이기 한다.

//기본적으로 동작하는 기능을 꺼야 하기에 disable()로 비활성화 한다.

@Bean

SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

http

.cors(cors -> cors.disable());

.csrf(csrf -> csrf.disable());

비활성화

http

.formLogin(login -> login.disable());

return http.build();

}



spring

- 회원가입 시 비밀번호를 암호화 하라
 - MemberService에 멤버필드로 PasswordEncoder를 선언하고 의존 객체를 자동 주입 한다.

```
@Service
public class MemberService {

    // id중복체크, 성공, 실패 상수변수 정의
    // 회원가입의 중복을 확인하는 상수
    public final static int user_id_alreday_exit = 0;
    // 회원가입의 성공여부를 확인하는 상수
    public final static int user_id_success = 1;
    // 회원가입의 실패를 확인하는 상수
    public final static int user_id_fail = -1;

    //MemberDAO도 DI를 정의한다.
    @Autowired
    MemberDAO memberdao;

    //암호화 하는 PasswordEncoder도 의존성 객체로 주입한다.
    @Autowired
    PasswordEncoder passwordencoder;
```




spring

- 회원가입 시 비밀번호를 암호화 하라
 - 비밀번호(pw)를 암호화한다.

```
//회원가입이 제대로 되었는지, 혹은 회원가입이 실패했는지 예외처리
public int signupConfirm(MemberDTO mdto) {
    System.out.println("MemberService signupConfirm()메소드 확인");

    //회원가입 중복체크
    //id 없음 => false
    boolean isMember = memberdao.isMember(mdto.getId());

    //회원가입 중복체크 통과했다면
    if(isMember == false) {
        // 중복된 아이디가 존재하지 않을 때 DB에 회원의 정보가 추가된다.
        // insertMember(mdto)부분에 암호화 하여 DAO로 넘긴다.
        // pw를 암호화한 비밀번호로 반환해준다.
        String encodePw = passwordencoder.encode(mdto.getPw());

        // 암호화된 encodePw를 mdto.setPw()에 넣어준다.
        mdto.setPw(encodePw);

        // insertMember(mdto)의 pw는 암호화된 비밀번호가 추가된다.
        // 즉, MemberDAO의 insertMember(mdto)에 암호화된 비밀번호가 추가된다.
        int result = memberdao.insertMember(mdto);
        if(result > 0 ) {
            return user_id_success; // result = 1
        }else {
            return user_id_fail; // result = -1
        }
    }
}
```



spring

- 회원가입 시 **비밀번호**를 암호화 하라
 - 회원가입을 진행한다. 단, 암호화 하면 비밀번호가 길어 짐을 주의한다.

전체 회원 목록

번호	아이디	비밀번호	Email	Phone	가입일	수정일
1	kjb1045	1111	k10@naver.com	1234	2026-01-26 05:39:00	2026-01-26 05:39:00
2	park	2222	park@naver.com	2345	2026-01-26 05:46:28	2026-01-26 05:46:28
3	kjb1045	1234	k10@naver.com	1234	2026-01-27 02:47:34	2026-01-27 02:47:34
4	lee123	123	le@naver.com	123	2026-01-27 03:45:55	2026-01-27 03:45:55
5	cha123	123	c123@naver.com	123	2026-01-27 03:50:16	2026-01-27 03:50:16
10	kjb1030	\$2a\$10\$gxAbJ6oDetoRBtpyrDJ5iO717iG98.ogI7FLqPO6fgs1HxnHwce	k1030@naver.com	010-1111-1111	2026-01-29 02:36:26	2026-01-29 02:36:26

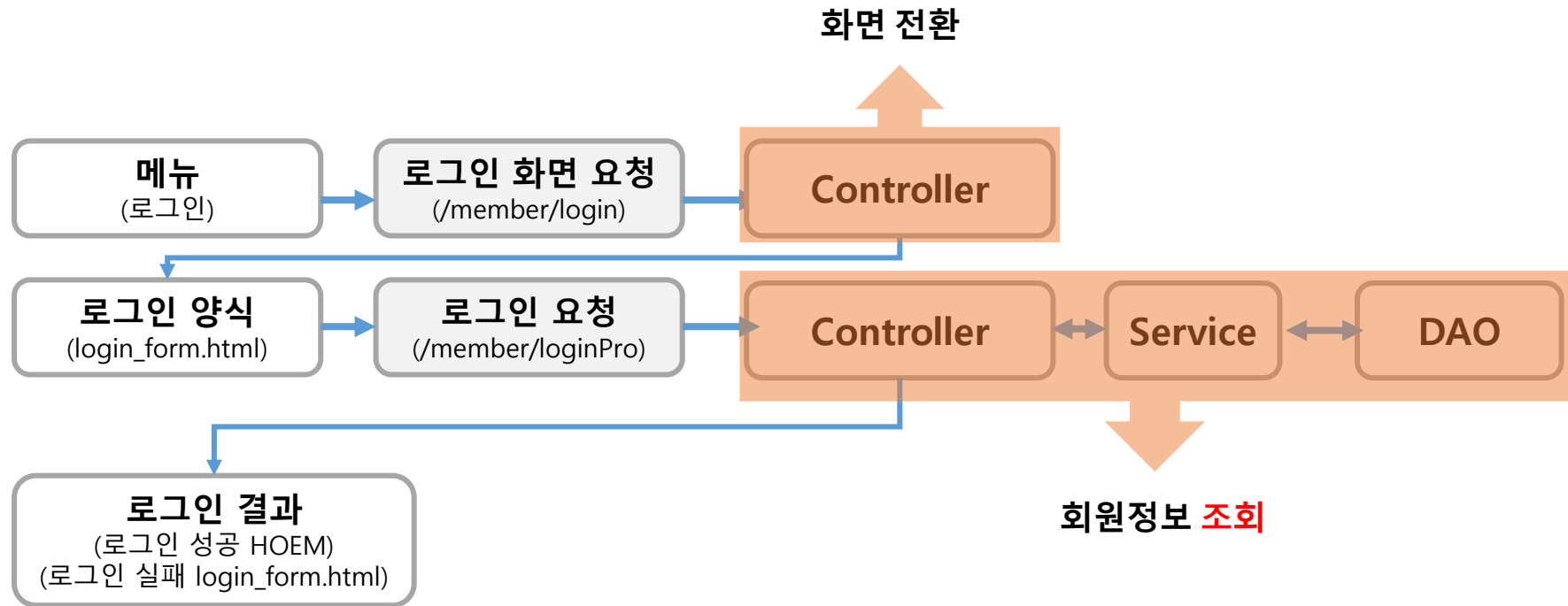
사용자 로그인 기능





spring

➤ 사용자 로그인 기능 구현 가이드





spring

➤ 사용자 로그인 기능 구현 가이드

- com.green -> src/main/resource -> member -> login_form.html 파일을 생성 후 작성한다.

```
<section>
  <div id="section_wrap">
    <div class="word">로그인</div>
    <div class="content">
      <form th:action="@{/member/loginPro}" method="post">
        <table width="400" border="1">
          <tr height="50">
            <td width="120" align="center">아이디</td>
            <td width="280" align="center">
              <input type="text" name="id" placeholder="아이디를 입력하세요" required>
            </td>
          </tr>
          <tr height="50">
            <td align="center">비밀번호</td>
            <td align="center">
              <input type="password" name="pw" placeholder="비밀번호를 입력하세요" required>
            </td>
          </tr>
          <tr height="50">
            <td align="center" colspan="2">
              <input type="submit" value="로그인">
              <input type="button" value="회원가입" th:onclick="location.href=@{/member/signup}" />
            </td>
          </tr>
        </table>
      </form>
    </div>
  </div>
</section>
```



spring

- **MemberService (로그인 로직 추가)**
- 암호화된 비밀번호를 비교하는 loginConfirm 메소드를 추가한다.
- 암호화된 비밀번호(PasswordEncoder)는 `dbPass.equals(inputPass)` 처럼 직접 비교할 수 없고, 반드시 `passwordEncoder.matches(평문, 암호문)` 메소드를 사용해야 한다는 점이 포인트이다.

```
public MemberDTO loginConfirm(MemberDTO mdto) {
    System.out.println("MemberService loginConfirm()메소드 확인");

    // 1. DB에서 해당정보의 Id 가져오기
    MemberDTO dbMember = memberdao.oneSelectMember(mdto.getId());

    // 2. 사용자가 존재하고 비밀번호가 일치하는지 확인
    // passwordEncoder.matches(사용자가 입력한 평문, DB에 저장된 암호문)
    if(dbMember != null && dbMember.getPw() != null) {
        if(passwordencoder.matches(mdto.getPw(), dbMember.getPw())) {
            return dbMember; // 로그인 성공시 회원정보 반환
        }
    }
    return null; //로그인 실패시 null 반환
}
```



spring

➤ MemberController

```
// 로그인 양식 폼 이동
@GetMapping("/member/login")
public String loginFrom() {
    System.out.println("MemberController loginFrom() 폼 이동");
    return "member/login_form";
}
```

- 로그인 성공하면 home으로, 실패하면 login 화면으로 이동하는 로그인 처리 컨트롤러 이다.

```
// 로그인 확인 처리
@PostMapping("/member/loginPro")
public String loginPro(MemberDTO mdto, RedirectAttributes re, HttpSession session) {
    System.out.println("MemberController loginPro() 처리");

    MemberDTO loggedMember = meberservice.loginConfirm(mdto);

    if (loggedMember != null) {
        // 로그인 성공 시 세션에 회원 정보 저장
        session.setAttribute("loggedMember", loggedMember);
        re.addFlashAttribute("msg", loggedMember.getId() + "님 환영합니다!");
        return "redirect:/";
    } else {
        // 로그인 실패 시
        re.addFlashAttribute("msg", "아이디 또는 비밀번호가 일치하지 않습니다.");
        return "redirect:/member/login";
    }
}
```



spring

➤ login_form.html 출력화면

로그인

아이디	아이디를 입력하세요
비밀번호	비밀번호를 입력하세요
로그인	회원가입

로그인성공

로그인실패

전체 회원 목록

번호	아이디	비밀번호	Email	Phone	가입일	수정일
1	kjb1045	1111	k10@naver.com	1234	2026-01-26 05:39:00	2026-01-26 05:39:00
2	park	2222	park@naver.com	2345	2026-01-26 05:46:28	2026-01-26 05:46:28
3	kjb1045	1234	k10@naver.com	1234	2026-01-27 02:47:34	2026-01-27 02:47:34
4	lee123	123	le@naver.com	123	2026-01-27 03:45:55	2026-01-27 03:45:55
5	cha123	123	c123@naver.com	123	2026-01-27 03:50:16	2026-01-27 03:50:16
10	kjb1030	\$2a\$10\$gxAbJ6oDetoRBtpyrDJ5iO717i7G98.ogI7FL0qPO6fgs1HxnHwce	k1030@naver.com	010-1111-1111	2026-01-29 02:36:26	2026-01-29 02:36:26

localhost:8090 내용:

아이디 또는 비밀번호가 일치하지 않습니다.

확인



spring

- login_form.html 에서 로그인 성공시 상단 메뉴줄에 [누구님]환영 합니다. 로그아웃 출력 되도록 src/main/resources -> templates -> include -> header_nav_footer.html의 내용을 아래처럼 수정한다.

```
</div>
<div id="header_wrap">
  <a th:href="@{/}">HOME</a>
  <!-- <a th:href="@{/member/signup}">회원가입</a>
  <a th:href="@{/member/login}">로그인</a> -->

  <th:block th:if="${session.LoginedMember == null}">
    <a th:href="@{/member/signup}">회원가입</a>
    <a th:href="@{/member/login}">로그인</a>
  </th:block>

  <th:block th:if="${session.LoginedMember != null}">
    <span style="font-weight: bold; color: #2c3e50;">
      <span th:if="${session.LoginedMember.id == 'admin1234'}" style="color: blue;">관리자</span>

      <span th:unless="${session.LoginedMember.id == 'admin1234'}">
        <span th:text="${session.LoginedMember.id}" style="color: #e74c3c;"></span>님 환영합니다!
      </span>
    </span>
    <a th:href="@{/member/logout}">로그아웃</a>

    <th:block th:if="${session.LoginedMember.id == 'admin1234'}">
      <a th:href="@{/member/list}" style="color: blue; font-weight: bold;">[회원목록]</a>
    </th:block>

    <a th:href="@{/member/memberInfo(id=${session.LoginedMember.id})}">내정보</a>
  </th:block>

  <a th:href="@{/board/list}">게시판</a>
</div>
```



spring

- login_form.html 출력화면에서 관리자 admin1234로 로그인 하면 아래메뉴처럼 출력 된다.

MEMBER JOIN	
HOME 관리자 로그아웃 [회원목록]	내정보 게시판
EXAMPLE	

- login_form.html 출력화면에서 관리자가 아닌 일반회원이 로그인 하면 아래메뉴처럼 출력 된다.

MEMBER JOIN	
HOME kjb1030님 환영합니다!	로그아웃 내정보 게시판
EXAMPLE	



spring

- **MemberController**
- 로그아웃 성공하면 HOME 화면으로 이동하는 컨트롤러 이다.

```
// 로그아웃
@GetMapping("/member/logout")
public String logout(HttpSession session, RedirectAttributes re) {

    // 1. 세션 무효화 (세션에 담긴 모든 데이터 삭제)
    session.invalidate();

    // 2. 로그아웃 완료 메시지 전달 (선택사항)
    re.addFlashAttribute("msg", "로그아웃 되었습니다.");

    // 3. 홈 화면으로 리다이렉트 (url: /)
    return "redirect:/";
}
```